

Entry number: 1

Date: 2026-09-25

AI tool/model: ChatGPT

Task ID/Title: Task #1.1 - Front End Signing up Feature

Purpose of AI Use: To get a detailed protocol on how to complete the signup page.

Chat Link or Prompt/Response: See Appendix

AI-Suggested Content:

Provided guidance on structuring the signup page using the following: React state, Zod validation and Supabase Auth. It also explained how to validate the user inputs, handle loading states and signup errors, store user's name as metadata, and navigate from signup to login page and vice-versa.

Validation:

Reviewed the response against the project requirements. Tested the signup page locally to verify if implementation was successful.

Decision:

Accepted – Most of the guidance was implemented

Reflection:

AI gave a structured series of steps to follow without giving the required code itself. This allowed me to learn and use its resources effectively.

Responsible Person: Daksikan Navanithan

Appendix:

AI prompt:

Hello, I am responsible for implementing the signup page for our CareerConnect project. We are using Next.js with TypeScript, Supabase for authentication, Zod for validation, and shadcn/ui components. Can you walk me through how I should implement the signup page myself?

The page needs four inputs: full name, email, password, and confirm password. Explain what React states I will need, how the form submission should work, how to validate the information with Zod before sending anything to Supabase, and how to display validation errors under the appropriate fields.

Also explain how I should use the existing Supabase client to create the account, including how to store the user's full name in the account metadata. Explain how I should handle Supabase errors, loading states, and what should happen after successful account creation.

Give also the implementation process and important TypeScript/Next.js functions or methods I will need, with small examples. Do not write the complete signup page for me. I want to use your explanation to implement the page myself.

AI response:

Since you want to implement the signup page yourself, I would break it into a few pieces: **form state** → **validation** → **Supabase signup** → **error handling** → **navigation**.

1. Make the page a client component

Your signup page needs React state and event handlers, so it should be a Next.js client component. At the top of the file, use:

```
"use client";
```

You will need `useState` to store the information entered into the form and `FormEvent` for the form submission handler.

The main pieces of state you will need are:

```
name
email
password
confirmPassword
errors
loading
```

The first four contain the user's inputs. `errors` can hold validation or authentication errors, while `loading` can be used to prevent the user from submitting the form multiple times while Supabase is processing the request.

2. Validate the information with Zod

Before sending anything to Supabase, validate the user's inputs locally.

Your `signupSchema` should validate an object containing:

```
{
  name,
  email,
  password,
  confirmPassword
}
```

For example, the schema should verify that:

- the name is not empty;
- the email has a valid email format;
- the password meets your project's minimum password requirement;
- the confirmation password is provided;
- password and `confirmPassword` are identical.

Zod's `.refine()` is useful for the last requirement because it lets you compare two fields.

Your schema could therefore follow this general structure:

```
const signupSchema = z
  .object({
    name: ...,
    email: ...,
    password: ...,
    confirmPassword: ...,
  })
  .refine((data) => data.password === data.confirmPassword, {
    message: "Passwords do not match",
    path: ["confirmPassword"],
  });
```

I would recommend keeping this schema in a separate validation file such as:

```
src/lib/validations/auth.ts
```

Then your signup page can simply import `signupSchema`.

3. Handle the form submission

Your `<form>` should call a function such as `handleSubmit` when submitted.

The first thing the function should do is prevent the browser's default form submission:

```
event.preventDefault();
```

Then validate the current form values:

```
const result = signupSchema.safeParse({
  name,
  email,
  password,
  confirmPassword,
});
```

`safeParse()` is useful here because it doesn't throw an exception when validation fails. Instead, you can check whether the validation succeeded.

Your overall logic should look like:

```
graph TD
    A[User presses Sign Up] --> B[Prevent default submission]
    B --> C[Validate with Zod]
    C --> D{Valid?}
    D -- No --> E[Show errors]
    D -- Yes --> F[Contact Supabase]
```

4. Display validation errors

If `safeParse()` fails, go through the Zod issues and associate each issue with its corresponding form field.

For example, if the email is invalid, you want the message to appear underneath the email input rather than showing one generic error at the top of the page.

Conceptually, your errors could be stored like:

```
{
  email: "Enter a valid email address",
  password: "Password must be at least 6 characters",
  confirmPassword: "Passwords do not match"
}
```

Then underneath an input you can conditionally display its error.

For example:

```
{errors.email && <p>{errors.email}</p>}
```

The important part is that **Supabase should not be called if Zod validation fails**.

5. Create the Supabase client

Once validation succeeds, use the Supabase browser client that already exists in your project.

For example:

```
const supabase = createClient();
```

You should import `createClient` from your existing Supabase client utility rather than configuring another Supabase client directly inside the signup page.

6. Create the account with Supabase Auth

Supabase provides:

```
supabase.auth.signUp()
```

At minimum, you'll provide:

```
email  
password
```

However, CareerConnect also needs the user's full name.

Supabase allows additional user information to be passed through the `options.data` property during signup. Therefore, the structure should conceptually be:

```
supabase.auth.signUp({  
  email,  
  password,  
  options: {  
    data: {  
      // user's name  
    },  
  },  
});
```

This lets the account be created using the email/password while also attaching the user's name as metadata.

7. Handle Supabase errors

The signup operation can return an error, so don't assume account creation always succeeds.

Your logic should check the returned `error`:

```
const { error } = await supabase.auth.signUp(...);
```

If an error exists, display an appropriate message to the user rather than redirecting them.

This could cover situations such as:

- an account already existing;
- invalid signup information;
- authentication configuration problems;
- network/server errors.

You don't necessarily need a separate custom message for every possible Supabase error during your first implementation. You can initially display the useful error message returned by Supabase.

8. Add a loading state

Set `loading` to `true` before calling Supabase and return it to `false` when the operation finishes.

While loading, you can disable the signup button and change its text:

`Sign up`

to something such as:

`Creating account...`

This prevents someone from clicking the button repeatedly and creating multiple signup requests.

A `try/finally` structure can be useful because `finally` allows you to reset the loading state regardless of whether signup succeeds or fails.

9. Redirect after successful signup

Since you're using Next.js, you can use:

```
const router = useRouter();
```

After Supabase successfully creates the account, call:

```
router.push(...)
```

with whichever route your team has decided users should reach after signup.

Exactly where you redirect depends on your authentication flow. For example, if email confirmation is required, you might show a confirmation message instead of immediately sending the user to an authenticated dashboard.

10. Add navigation to login

At the bottom of the signup form, users who already have an account should be able to reach the login page.

Since this is internal Next.js navigation, use:

```
<Link href="/auth/login">
```

rather than a normal HTML `<a>` element.

You could display something like:

Already have an account? [Log in](#)

11. Use your existing UI components

Since your project already uses `shadcn/ui`, you don't need to create your own input and button components.

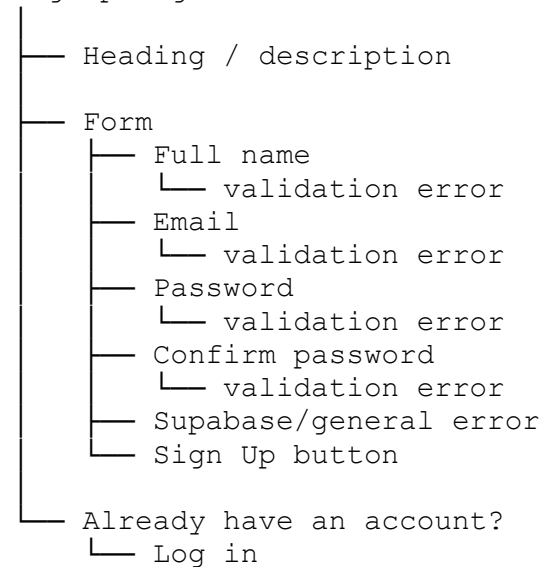
You can use:

```
<Input />
<Label />
<Button />
```

for the four fields and submission button.

Your final page structure will therefore roughly be:

Signup Page



A couple of optional improvements

Once the basic signup functionality works, you could also consider:

- a **show/hide password** button;
- a visual **password-strength indicator**;

- clearing an individual field's error once the user corrects it;
- disabling the form while the signup request is running.

I wouldn't prioritize those until the required Sprint functionality is working.

For your first implementation, focus on getting this flow working correctly:

```
Enter information
  ↓
Zod validation
  ↓
Display field errors if invalid
  ↓
Supabase Auth signUp()
  ↓
Handle Supabase error
  ↓
Successful account creation
  ↓
Navigate to the appropriate next page
```

Once that works, you can improve the user experience separately.